

NetSuite Native Integration

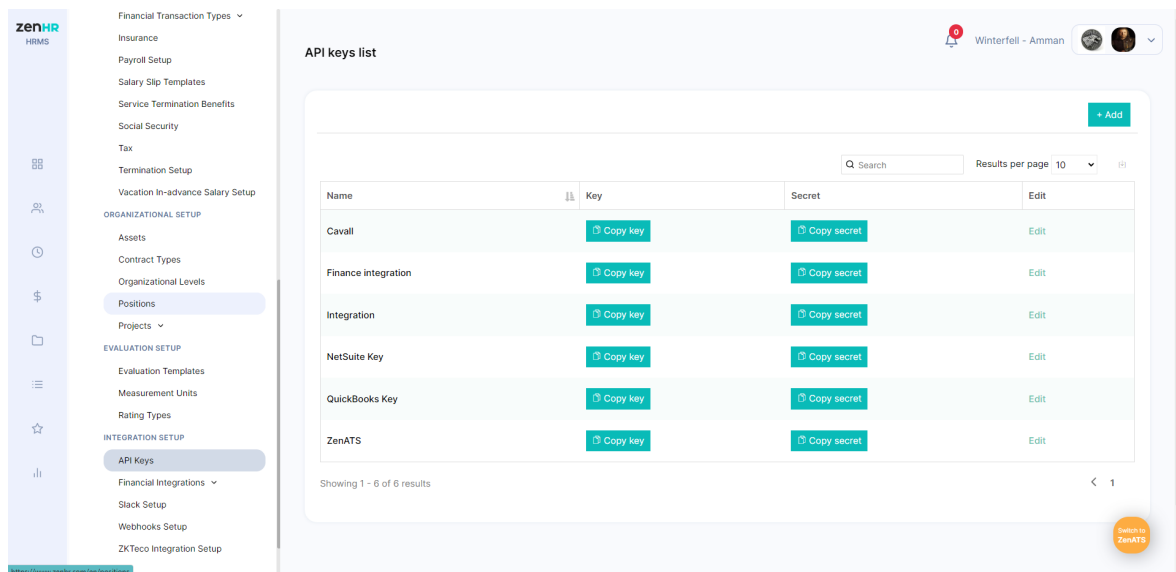
ZenHR's native integration with NetSuite allows customers to push/transfer their employees' data and the calculated salaries from ZenHR to NetSuite to gain all the benefits that come with the most user-friendly HRMS, ZenHR.

How to Activate This Integration

To activate NetSuite's native integration on ZenHR, you first have to generate the needed API keys to proceed with the integration process.

To generate the API keys, follow these steps:

- Click your user icon in the upper right corner and go to **System Preferences** → **Integration Setup** → **API Keys**.



The screenshot displays the 'API keys list' page in the ZenHR system. On the left is a sidebar with navigation options like 'Financial Transaction Types', 'Insurance', 'Payroll Setup', etc., with 'API Keys' highlighted under 'INTEGRATION SETUP'. The main area shows a table of existing API keys. Each entry includes its name, a copyable key, a copyable secret, and an edit option. A '+ Add' button is located at the top right of the table area.

Name	Key	Secret	Edit
Cavall	Copy key	Copy secret	Edit
Finance integration	Copy key	Copy secret	Edit
Integration	Copy key	Copy secret	Edit
NetSuite Key	Copy key	Copy secret	Edit
QuickBooks Key	Copy key	Copy secret	Edit
ZenATS	Copy key	Copy secret	Edit

- On the **API keys list** page, click the “Add” button.
- Add the API key **Name** and **Permissions**.
- Click “Create” to create the NetSuite API key.

Add API key

Name*

Permissions ⓘ *

IP addresses ⓘ

Use ☒ Default ☐ OAuth1 ☐ OAuth2

[Create](#)

- Then click **“Edit”** beside the API.

[+ Add](#)

Results per page 10 ⓘ

Name	Key	Secret	Edit
NetSuite Key	Copy key	Copy secret	Edit

Showing 1 - 1 of 1 results (filtered from 6 total entries) < 1

- Activate **“OAuth1”**.
- On NetSuite, go to **Setup → Company → Enable Features → SuiteCloud** and tick the option **“Token Based Authentication”** from the section **“Manage Authentication.”**

Manage Authentication

☐ SUITESIGNON
USE NETSUITE AS THE TRUSTED SYSTEM TO AUTHENTICATE ACCESS TO INTEGRATED EXTERNAL APPLICATIONS THROUGH SINGLE SIGN-ON. BY ENABLING THIS FEATURE, YOU AGREE TO [SUITECLOUD TERMS OF SERVICE](#)

☐ OPENID CONNECT (OIDC) SINGLE SIGN-ON
ENABLE OPENID CONNECT (OIDC) AS AN ADDITIONAL AUTHENTICATION MECHANISM FOR YOUR USERS. BY ENABLING THIS FEATURE, YOU AGREE TO [SUITECLOUD TERMS OF SERVICE](#)

☐ SAML SINGLE SIGN-ON
ENABLE SAML AS AN ADDITIONAL AUTHENTICATION MECHANISM FOR YOUR USERS. BY ENABLING THIS FEATURE, YOU AGREE TO [SUITECLOUD TERMS OF SERVICE](#)

☒ TOKEN-BASED AUTHENTICATION
ENABLE TOKEN-BASED AUTHENTICATION AS AN ADDITIONAL AUTHENTICATION MECHANISM FOR YOUR USERS. BY ENABLING THIS FEATURE, YOU AGREE TO [SUITECLOUD TERMS OF SERVICE](#)

- On NetSuite again, go to **Setup → Integrations → Manage Integrations**.

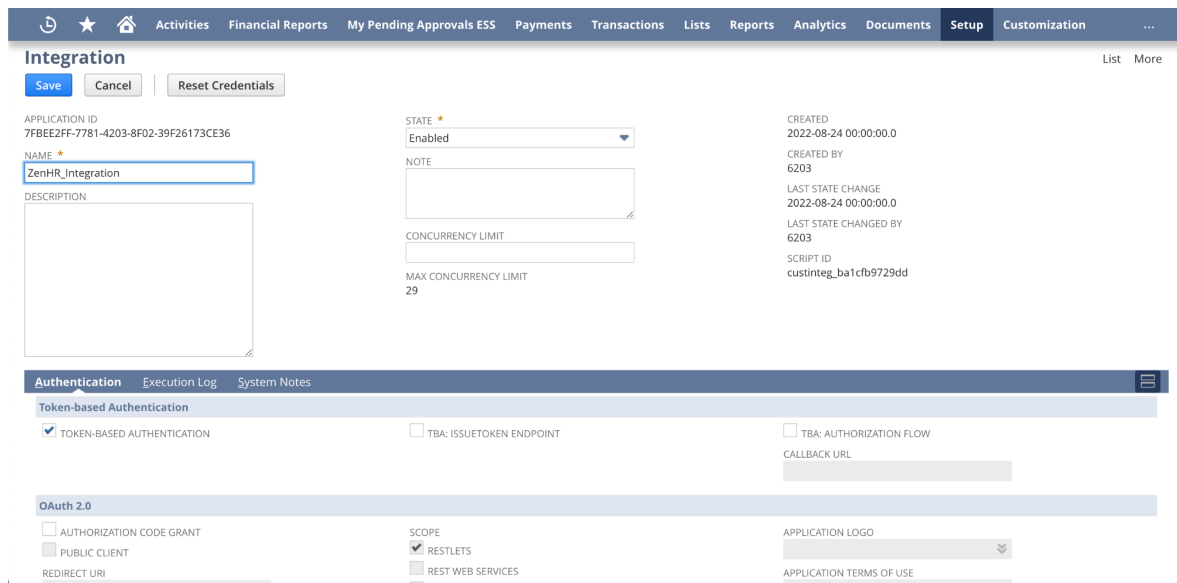
Integrations [Set Preferences](#)

[New](#) [Refresh](#)

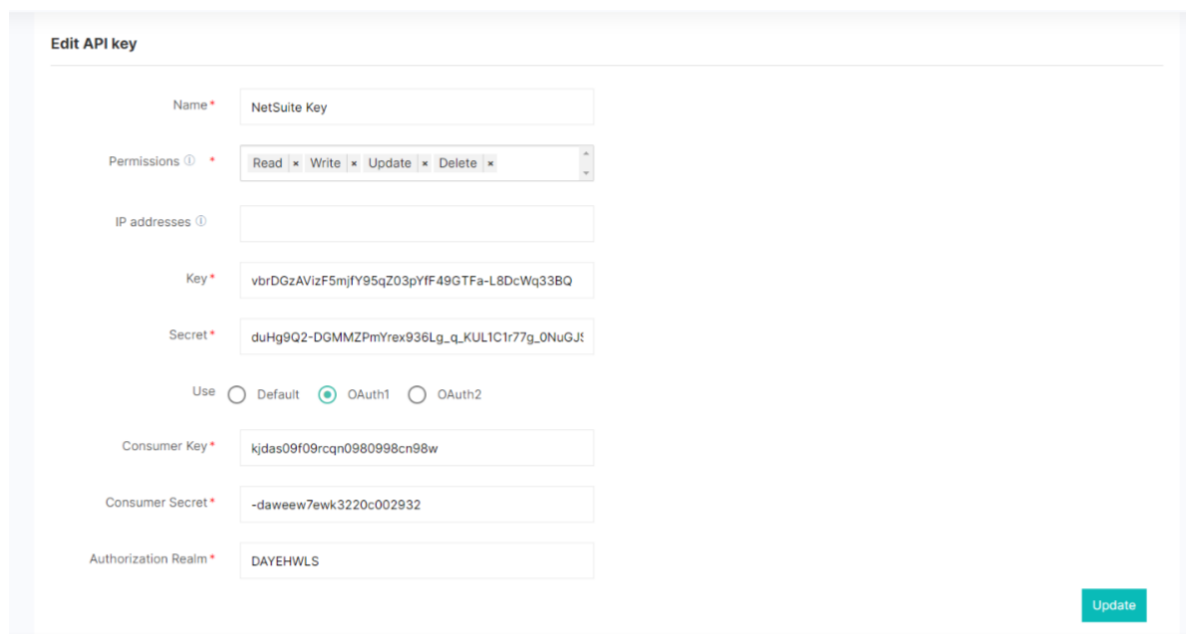
☐ SHOW INACTIVES

NAME	APPLICATION ID	STATE	CREATED ON
Default Web Services Integrations		Enabled	

- Click **“New”** to set up a new integration.



- Name the integration name “ZenHR_Integration” and make sure to select the **State: Enabled**, tick the option “**Token-based authentication**”, select the **Scope: RESTLETS**, and uncheck the options “**TBA: ISSUETOKEN ENDPOINT**”, “**TBA: AUTHORIZATION FLOW**”, “**AUTHORIZATION CODE GRANT**”, and “**USER CREDENTIALS**”.
- A **Consumer Key** and **Consumer Secret** will be generated. Make sure that you copy and save them as they only appear once.
- To get the **Authorization Realm** from NetSuite, go to **Setup → Company → Company information**, and copy the **Account ID**.
- Copy the **Client ID** and **Client Secret** to your API Key setup in ZenHR.



NetSuite: Creating New User With Role

- On NetSuite, go to **Setup** → **Users/Roles** → **Manage Roles**, and create a new role.

Role ← → List Search Copy to Account System Notes More

Save **Cancel** **Change ID** **Actions**

General

NAME *
ZenHR Admin

ID
customrole1056

CENTRE TYPE
Accounting Centre

EMPLOYEE RESTRICTIONS
none - no default ☐ ALLOW VIEWING

☐ DO NOT RESTRICT EMPLOYEE FIELDS
☐ RESTRICT TIME AND EXPENSES
☐ SALES ROLE
☐ SUPPORT ROLE
☐ INACTIVE
☒ CORE ADMINISTRATION PERMISSIONS

Subsidiary Restrictions

ACCESSIBLE SUBSIDIARIES
☒ ALL
☐ ACTIVE
☐ USER SUBSIDIARY
☐ SELECTED
☐ ALLOW CROSS-SUBSIDIARY RECORD VIEWING

Authentication

- Make sure that the user can access all subsidiaries.
- The user can list accounts, departments, employee records, and employees.

Permissions	Restrictions	Forms	Searches	Users	Preferences	Dashboard	Translation	History
Transactions •	Reports	Lists •	Setup •	Custom Record				
PERMISSION *	LEVEL							
Accounts	Full							
Departments	Full							
Employee Record	Full							
Employees	Full							

- From the **Setup**, make sure that the user has full permission on the user access token.

Permissions	Restrictions	Forms	Searches	Users	Preferences	Dashboard	Translation	History
Transactions •	Reports	Lists •	Setup •	Custom Record				
PERMISSION *	LEVEL							
User Access Tokens	Full							

- From the **Transactions**, make sure that the user has full permission to make journal entries.

Permissions	Restrictions	Forms	Searches	Users	Preferences	Dashboard	Translation	History
Transactions •	Reports	Lists •	Setup •	Custom Record				
PERMISSION *	LEVEL							
Make Journal Entry	Full							

- Link the user that you will use for the integration with this role, noting that the user should not have an administrator role on NetSuite.

NetSuite: Creating New Access Token

- Go to **Setup** → **Users/Roles** → **Access Token** and create a new access token.
- Select the integration **Application Name**, the **User**, and the **Role**.
- Insert the **Token Name**.
- Then click **“Save”**.
- Copy the generated token details and keep them with you, as they will only be generated once.

Access Token

Primary Information

APPLICATION NAME *

USER *

ROLE *

TOKEN NAME *

☐ INACTIVE

NetSuite: Running The Scripts

- Go to **Customization** → **Scripting** → **Scripts** and create a new script.

Upload Script File

SCRIPT FILE *

File

ATTACH FROM *

FILE NAME

FOLDER *

URL

SELECT FILE
 No file chosen

CHARACTER ENCODING

☐ INACTIVE
☐ AVAILABLE FOR SUITEBUNDLES
☐ HIDE IN SUITEBUNDLE
☐ AVAILABLE WITHOUT LOGIN
☐ COMPANY-WIDE USAGE
☐ GENERATE URL TIME STAMP

- Upload the scripts from your computer and then deploy each with **Status: Released** and **Log Level: Debug**.

NetSuite Scripts

You can find and download the scripts through the link: [NetSuite Scripts](#)

GL Accounts Script

```
/**
 *@NApiVersion 2.1
 *@NScriptType Restlet
 */
define(['N/search'], function(search) {
  function _get() {
    try{
      var searchQuery = search.create( {
        type: search.Type.ACCOUNT,
        columns: ['name', 'isinactive', 'type', 'subsidiary']
      });

      var resultSet = getAllResults(searchQuery)

      log.debug("the accounts count: ", resultSet.length)
      return resultSet;
    }
    catch(exception){
      log.debug('SCRIPT STOPPED', exception);
      return {
        code : 404,
        message : 'Something went wrong! ' + exception
      };
    }
  }

  function getAllResults(searchQuery) {
    var results = searchQuery.run();
    var searchResults = [];
```

```

var searchId = 0;
do {
  var resultslice = results.getRange({ start: searchId, end: searchId + 1000 });
  resultslice.forEach(function(result) {
    searchResults.push({
      accountId: result.id,
      accountName: result.getValue('name'),
      isInactive: result.getValue('isinactive'),
      accountType: result.getValue('type'),
      subsidiaryId: result.getValue('subsidiary')
    });
    searchId++;
  });
} while (resultslice.length >= 1000);
return searchResults;
}

return {
  get: _get,
}
});

```

Employees Script

```

/**
 * @NApiVersion 2.0
 * @NModuleScope SameAccount
 * @NScriptType RESTlet
 */

define(["N/record", "N/search", "N/log"],
function (record, search, log) {
  var exports = {};
  const options = new Array("hireDate", "birthDate", "gender", "releaseDate", "maritalStatus",
"class", "department", "location", "supervisor", "subsidiary")

  function createOrUpdateEmployee(params) {

```

```

try {
  log.debug("create/update Employee start", "Script is running");
  log.debug("with params", params);

  var employee = getEmployee(params.custentityHRId);
  return (employee.id) ? updateEmployee(params, employee.id) : createEmployee(params);
} catch (error) {
  log.debug("something wrong", error);

  return { statusCode: 401, message: "something is wrong!" + error.message };
}
}

function getEmployee(employeeId) {
  log.debug("employee search", "Searching for existing employee");

  var employee = search.create({
    type: record.Type.EMPLOYEE,
    filters: [["custentity_ns_zenhr_employee_id", "is", String(employeeId)]]
  }).run().getRange({ start: 0, end: 1});
  return (!employee[0]) ? false : employee[0];
}

function createEmployee(params) {
  log.debug("create", "creating employee");

  var employee = record.create({
    type: record.Type.EMPLOYEE,
    isDynamic: true
  });
  return saveEmployee(employee, params);
}

function updateEmployee(params, employeeId) {
  log.debug("update", "updating employee");

  var employee = record.load({
    type: record.Type.EMPLOYEE,
    id: employeeId,
    isDynamic: true
  });
  return saveEmployee(employee, params);
}

function saveEmployee(employee, params) {

```



```

    for (var key in params) {
        if (params.hasOwnProperty(key)) {
            if (options.indexOf(key) >= 0) {
                employee.setText({ fieldId: key.toLowerCase(), text: params[key] });
            } else if (key == "custentityHRId") {
                employee.setText({ fieldId: "custentity_ns_zenhr_employee_id", text: String(params[key])
            });
        } else {
            employee.setValue({ fieldId: key.toLowerCase(), value: params[key] });
        }
    }
    log.debug("saving employee", employee)
    employee.save({ enableSourcing: true, ignoreMandatoryFields: true });
    return { statusCode: 201, message: "Saved Successfully!" };
}

exports.post = createOrUpdateEmployee;
return exports;
});

```

Employees Script

```

/**
 * @NApiVersion 2.0
 * @NModuleScope SameAccount
 * @NScriptType RESTlet
 */

define(["N/record", "N/search", "N/log"],
function (record, search, log) {
    var exports = {};

    function createJournalEntry(params) {
        try {
            log.debug("Create Journal Entries start", "Script is running");
            log.debug("with params", params);

            for (var index = 0; index < params.journals.length; index++) {
                log.debug("Creating Journal Entry", params.journals[index].name);

                var journalEntry = record.create({
                    type: record.Type.JOURNAL_ENTRY,

```

```

        isDynamic: true
    })

    journalEntry.setText({ fieldId: "trandate", text: params.journals[index].date });
    journalEntry.setText({ fieldId: "subsidiary", text: "Group : " +
params.journals[index].subsidiary });
    journalEntry.setValue({ fieldId: "memo", value: params.journals[index].memo });

    for (var line = 0; line < params.journals[index].lines.length; line++) {
        log.debug("Adding Line", params.journals[index].lines[line])

        journalEntry.selectNewLine({ sublistId: "line" });
        for (var field in params.journals[index].lines[line]) {
            log.debug("Adding field", field)

            if (field == "paymentMethod" || field == "employeeInternalId") {
                continue
            } else if (field == "account" || field == "location" || field == "name") {
                journalEntry.setCurrentSublistText({ sublistId: "line", fieldId: field, text:
params.journals[index].lines[line][field], ignoreFieldChange: true });
            } else if (field == "department") {
                journalEntry.setCurrentSublistValue({ sublistId: "line", fieldId: field, value:
getDepartmentId(params.journals[index].lines[line][field]), ignoreFieldChange: true });
            } else if (field == "debit" || field == "credit") {
                journalEntry.setCurrentSublistValue({ sublistId: "line", fieldId: field, value:
params.journals[index].lines[line][field].toFixed(2), ignoreFieldChange: true });
            } else {
                journalEntry.setCurrentSublistValue({ sublistId: "line", fieldId: field, value:
params.journals[index].lines[line][field], ignoreFieldChange: true });
            }
        }
        journalEntry.commitLine({ sublistId: "line" });
    }
    log.debug("saving Journal Entry", journalEntry)

    journalEntry.save({ enableSourcing: true, ignoreMandatoryFields: true });
}
return { statusCode: 201, message: "Created Successfully!" };
} catch (error) {
    log.debug("something wrong", error);

    return { statusCode: 401, message: "something is wrong!" + error.message };
}
}

```

```
function getDepartmentId(departmentText) {  
  var department = search.create({  
    type: record.Type.DEPARTMENT,  
    filters: [["name", "is", departmentText]]  
  }).run().getRange({ start: 0, end: 1});  
  return (!department[0]) ? "" : department[0].id;  
}
```

```
exports.post = createJournalEntry;  
return exports;  
});
```

ZenHR: NetSuite Integration Setup

- On ZenHR, click your user icon in the upper right corner and go to **System Preferences** → **Integration Setup** → **Financial Integrations** → **NetSuite Setup**.
- To get the needed URLs from your NetSuite account:
 - Go to **NetSuite** → **Customization** → **Scripts** → **Script Deployments**
 - Click **“View”** in the needed script record.
 - Copy the **External URL** to add it to the ZenHR NetSuite integration setup.
- **Step 1:** Enter the API key that you created previously.
- **Step 2:** Activate the toggle in the **Employee**, **GL Account Identifiers**, and **Journal Entry** sections and add the **External URL** retrieved from your NetSuite account.

NetSuite Setup

System Preferences / Integrations / NetSuite Setup

Api key *

NetSuite Key 

Employee

Activate



URL *

https://12345.restlets.api.netsuite.com/app/site/hosting

GL Account Identifiers

Activate



URL *

https://12345.restlets.api.netsuite.com/app/site/hosting

Journal Entry

By activating NetSuite integration, you need to add the Subsidiary ID from your NetSuite account to your branches setup in ZenHR to link your accounts

Activate



URL *

https://12345.restlets.api.netsuite.com/app/site/hosting



- Make sure that the branch name on ZenHR and the subsidiary name on NetSuite match.
- Make sure that the organization levels (*work location, departments, etc.*) match on both ZenHR & NetSuite.
- Using the following options, set up the way that your salaries' journal entries will be created and how they'll match the voucher.

Journal Entry

By activating NetSuite integration, you need to add the Subsidiary ID from your NetSuite account to your branches setup in ZenHR to link your accounts

Activate ☒

URL *

Journal maximum number of employees

Use clearance accounts to match debit & credit ☒

Generate new journal for company contributions ☒

Auto generate journal for off-cycle payments ☐

Auto generate journal for off-cycle vacation in advance salaries ☒

Calculate employees' provision ☒

Generate a new journal for employees' provision ☒

Generate a new journal for company expenses ☒

Journal type ☒ Summary ☐ Detailed

Group by

Create new journal for each group ☒

[Save](#)

[Events Viewer](#) [Switch to ZenATS](#)

Then, you will be able to set up the way that your salaries journal will be created and how the voucher will match, with the following options:

- Use clearance accounts to match debit & credit: **Yes** or **No**.
- Generate new journal for company contributions: **Yes** or **No**.
- Journal type: **Summary** or **Detailed**
- Group by (If Journal type: **Summary** was selected).
- Create a new journal for each group (If Journal type: **Summary** was selected).

NetSuite: Adding Customized Fields

- On NetSuite, go to **Customization** → **Lists Records & Fields** → **Custom Entity Fields** from your NetSuite account, and click on new.

Custom Entity Field

[Save](#) [Cancel](#) [Change ID](#) [Apply to Forms](#) [Actions](#)

Label *

ID

OWNER

DESCRIPTION

TYPE

LIST/RECORD

☒ STORE VALUE ☐ USE ENCRYPTED FORMAT

☐ SHOW IN LIST

☐ GLOBAL SEARCH

☐ RECORD IS PARENT

☐ INACTIVE

[←](#) [→](#) [List](#) [Copy to Account](#)

Applies To **Display** **Validation & Defaulting** **Sourcing & Filtering** **Access** **Translation** **History**

☐ CUSTOMER

☐ VENDOR

☒ EMPLOYEE

☐ OTHER NAME

☐ CONTACT

☐ AGENT

☐ WEB SITE

☐ GROUP

☐ AVAILABLE EXTERNALLY

☐ PRINT ON STATEMENT

☐ PRINT ON PRICE LIST

☐ PROJECT TEMPLATE

[Save](#) [Cancel](#) [Change ID](#) [Apply to Forms](#) [Actions](#)

- Insert the label “zenhr_employee_id”, and the ID will be generated as “ns_zenhr_employee_id.”

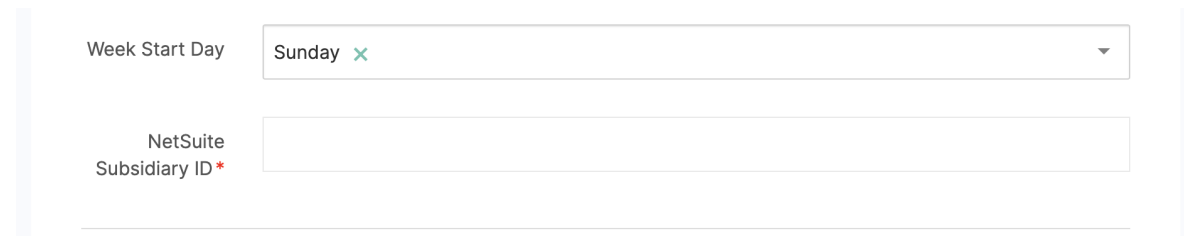
- Select the type “**Integer**”.
- Tick the option “**Store Value**”.
- Select the option **Employees** from the section **Applies To**.

Note: Before activating the employees' integration, make sure to insert ZenHR employees' internal ID in the new custom field.

Adding NetSuite Subsidiary ID

To link your branch to the correct NetSuite account:

- On NetSuite, go to **Setup** → **Company** → **Subsidiaries list**.
- Find the Subsidiary ID.
- Then, on ZenHR, click your user icon in the upper right corner and go to **System Preferences** → **General** → **Branch Setup**.
- Add the **Subsidiary ID**



Week Start Day Sunday 

NetSuite Subsidiary ID *

Adding NetSuite GL Identifiers:

To add Payroll GL Accounts:

- On ZenHR, click your user icon in the upper right corner and go to **System Preferences** → **Financial Setup** → **Payroll Setup**.
- Enter the relevant details under **GL Accounts**.
- Then click “**Update**”.

- GL Accounts

General Ledger (GL) account numbers are the names and numbers of all the bookkeeping accounts a business uses to keep track of its financial transactions and to prepare financial reports. During general ledger integration, the items updated on the current accounts are transferred to the corresponding general ledger accounts.

Social Security

Employee GOSI

Company GOSI

Company social security clearance account

Health Insurance

Employee health insurance

To add Financial Transactions GL Accounts:

- Click your user icon in the upper right corner and go to **System Preferences** → **Financial Setup** → **Financial Transaction Types** → **Income & Deductions**.
- Open the **Financial Transaction Type** by clicking “**Edit**” on the **Transaction Type** record.
- Enter the **Amount**, **GL Account Number**, **Clearance account**, **Minimum**, and **Maximum**.
- Then click “**Create Financial Transaction Type**”.

Amount *

-

%

+

GL Account Number

Clearance account

Minimum

Maximum

Create Financial Transaction Type

Switch to ZenATS

To add Loans GL Accounts:

- Click your user icon in the upper right corner and go to **System Preferences** → **Financial Setup** → **Financial Transaction Types** → **Loan Types**.
- Open the **Loan Type** by clicking “**Edit**” on the **Loan Type** record.
- Enter **GL Identifier** and **Clearance account**.

- Then click **“Update”**.

GL Identifier

GL identifier

Clearance account

Update Delete

To add Overtime GL Accounts

- Click your user icon in the upper right corner and go to **System Preferences → Financial Setup → Financial Transaction Types → Overtime Types**
- Open the **Overtime Type** by clicking **“Edit”** on the **Overtime Type** record.
- Enter **GL Identifier** and **Clearance account**.
- Then click **“Update Overtime Type”**.

GL identifier

Clearance account

Overtime Exceptions

+ Add Exception

Update Overtime Type

Delete

That’s it; your NetSuite integration will now be complete!

How to Use This NetSuite Integration

Now that you’ve successfully integrated NetSuite with your ZenHR account, you can now post salaries from ZenHR to Netsuite.

To post salaries from ZenHR to NetSuite, follow these steps:

- On ZenHR, on the left side of the page, go to **Main Menu → Payroll → Manage Financials → Manage Salaries**.
- Select and filter the salary year and month for all employees.
- Then click **“Send to NetSuite”**, and your salaries will be transferred as journals.

Payroll / Manage Salaries

Manage Salaries

Send to NetSuite Calculate

Salaries Salary Batches

Salaries

Show Filters ▼

NetSuite Failed Posts Events Viewer

- To retry sending failed posts, go to **User Menu** → **System Preferences** → **Integration Setup** → **Financial Integrations** → **NetSuite Setup**.
- On the **Events Viewer** page, click “**Retry**”.

Events Viewer

Results per page 10 

Event	Response Code	Resource Type	Action
64805	404	Journal Entry	Retry